SECTION A – DEBUGGING QUESTIONS

Question 1: Dangling Reference in Return-by-Reference

1. Explain what happens to the memory location of result after getMaximumScaled() finishes execution. Why does danglingRef exhibit undefined behavior?

- The variable result is a local variable stored in the stack memory.
- When getMaximumScaled() finishes execution, its stack memory is released.
- The reference returned by the function points to an invalid memory location.
- Therefore, accessing danglingRef causes undefined behavior and may produce garbage values or a segmentation fault.

2. Why is getArrayElement() safe, whereas getMaximumScaled() is unsafe?

- getArrayElement() returns a reference to an existing array element.
- The array is declared in the calling function and remains valid after the function returns.
- getMaximumScaled() returns a reference to a local variable that gets destroyed after function execution.
- Hence, returning an array element is safe, but returning a local variable by reference is unsafe.

3. Corrected C++ Code

```
#include <iostream>
using namespace std;

int getMaximumScaled(int a, int b, int scaleFactor) {
   int maxVal = (a > b) ? a : b;
   return maxVal * scaleFactor;
}

int& getArrayElement(int arr[], int index) {
   return arr[index];
}

int main() {
   int x = 10, y = 20;

   int scaledValue = getMaximumScaled(x, y, 2);
```

```
    cout << "Scaled Max: " << scaledValue << endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;

    cout << "Updated Array[1]: " << data[1] << endl;
    cout << "Scaled Value: " << scaledValue << endl;

    return 0;
}
```

4. Dry Run and Expected Output

Step	Operation	Result
1	x = 10, y = 20	10, 20
2	Maximum value	20
3	20 × 2	40
4	Update data[1]	999
5	Display scaled value	40

Output:

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value: 40
```

Question 2: Static Data Member Linkage & Member Function Scoping

1. Identify the linker error and explain why declaring a static member inside a class is insufficient.

- The linker error is undefined reference to BankAccount::totalBankReserves.

- A static data member is shared by all objects of a class.
- Declaring it inside the class only announces its existence.
- It must be defined outside the class to allocate memory.

2. Identify the compilation error on Line 19.

- The static member function displayAudit() cannot directly access the non-static variable accHolder.
- Non-static members belong to individual objects.
- Static functions do not have a this pointer.
- Therefore, an object is required to access individual account information.

3. Corrected C++ Code

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
   string accHolder;
   double balance;
   static double totalBankReserves;

public:
   BankAccount(string name, double initialDeposit) {
      accHolder = name;
      balance = initialDeposit;
      totalBankReserves += initialDeposit;
   }

   void displayAccount() {
      cout << "Account Holder: " << accHolder << endl;
      cout << "Balance: $" << balance << endl;
   }

   static void displayAudit() {
      cout << "Total Reserves in Vault: $"
           << totalBankReserves << endl;
   }
};

double BankAccount::totalBankReserves = 0.0;
```

```
int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();

    BankAccount::displayAudit();

    return 0;
}
```

4. Console Output

```
Account Holder: Alice
Balance: $5000
Account Holder: Bob
Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

1. Why does the compiler generate an ambiguous call error on Line 19?

The call computeVolume(5.0) creates ambiguity because two functions can accept one argument:

- Overload 1: computeVolume(double length, double width = 1.0, double height = 1.0)
- Overload 2: computeVolume(double side)

Both functions match the given argument. Therefore, the compiler cannot decide which function to execute.

2. Does the call on Line 22 also conflict?

Yes. The call computeVolume(3.0, 7.0) is also ambiguous.

- Overload 1 accepts two arguments because the third argument has a default value.
- Overload 3 accepts exactly two arguments.
- Both functions match the call, causing ambiguity.

3. Refactored Function Signatures

To eliminate ambiguity, remove the default arguments and use different parameter counts:

- Cuboid: Three arguments.
- Cube: One argument.
- Cylinder: Two arguments.

4. Corrected C++ Code

```
#include <iostream>
using namespace std;

double computeVolume(double length) {
   return length * length * length;
}

double computeVolume(double radius, double height) {
   return 3.14159 * radius * radius * height;
}

double computeVolume(double length, double width, double height) {
   return length * width * height;
}

int main() {
   cout << "Cube: " << computeVolume(5.0) << endl;
   cout << "Cylinder: " << computeVolume(3.0, 7.0) << endl;
   cout << "Cuboid: " << computeVolume(5.0, 4.0, 3.0) << endl;

   return 0;
}
```

Output:

```
Cube: 125
Cylinder: 197.92
Cuboid: 60
```

Question 4: Array of Objects Missing Default Constructor

1. Explain why Student batch[3] fails to compile.

- An array of objects requires a default constructor to initialize its elements when declared without initializers.
- The programmer has defined only a parameterized constructor.
- When a parameterized constructor is defined, the compiler does not automatically generate a default constructor.
- Therefore, Student batch[3] causes a compilation error.

2. Two techniques to solve this problem

Technique A: Explicit Default Constructor

Create a default constructor with no arguments to initialize the objects.

Technique B: Array Initializer List

Initialize each array element using the parameterized constructor.

Example:

```
Student batch[3] = {
   Student(101, "Aarav"),
   Student(102, "Bhavna"),
   Student(103, "Chirag")
};
```

3. Corrected C++ Code – Technique A

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
   int rollNumber;
   string name;

public:
   Student() {
      rollNumber = 0;
      name = "";
   }

   Student(int r, string n) {
      rollNumber = r;
```

```
        name = n;
    }

    void display() {
        cout << "Roll: " << rollNumber
             << ", Name: " << name << endl;
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; i++) {
        batch[i].display();
    }

    return 0;
}
```

4. Console Output

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration & Scope Resolution Flaws

1. Why can't alertThreshold be accessed directly? Does a friend function possess a this pointer?

- A friend function is a non-member function.
- It does not belong to any particular class.
- It has no implicit this pointer.
- Therefore, private members must be accessed through the objects passed as arguments.
- The correct access is a.alertThreshold.

2. Why does v8.inspectDiagnostics(v8, audit) fail?

- A friend function is not a member function.
- Therefore, it cannot be called using an object and dot operator.
- The correct syntax is:

inspectDiagnostics(v8, audit);

3. Why is a forward declaration of TelemetryAuditor required?

- The Engine class refers to TelemetryAuditor in its friend function declaration.
- Since TelemetryAuditor is defined later, the compiler must know that the class exists.
- Therefore, class TelemetryAuditor; must be declared before the Engine class.

4. Corrected C++ Code

```
#include <iostream>
using namespace std;

class TelemetryAuditor;

class Engine {
private:
   int temperature;
   int rpm;

public:
   Engine(int t, int r) {
      temperature = t;
      rpm = r;
   }

   friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

class TelemetryAuditor {
private:
   int alertThreshold;

public:
   TelemetryAuditor(int threshold) {
      alertThreshold = threshold;
   }
```

```
    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    if (e.temperature > a.alertThreshold) {
        cout << "Warning: Engine overheat!" << endl;
    } else {
        cout << "Engine operating nominally at "
             << e.rpm << " RPM." << endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);

    return 0;
}
```

Output:

Warning: Engine overheat!

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: Corporate Vault Management with Static Data & Functions

1. Structured Pseudocode

```
CLASS BranchOffice

PRIVATE:
    branchID : STRING
    branchName : STRING
    currentBalance : REAL
```

```
    STATIC totalVaultReserves : REAL = 0.0
    STATIC totalBranchesActive : INTEGER = 0

PUBLIC:

    CONSTRUCTOR(id, name, openingDeposit)
        branchID = id
        branchName = name
        currentBalance = openingDeposit

        totalVaultReserves = totalVaultReserves + openingDeposit
        totalBranchesActive = totalBranchesActive + 1
    END CONSTRUCTOR

    METHOD Deposit(amount)
        currentBalance = currentBalance + amount
        totalVaultReserves = totalVaultReserves + amount
    END METHOD

    METHOD Withdraw(amount) RETURNS BOOLEAN
        IF amount <= currentBalance THEN
            currentBalance = currentBalance - amount
            totalVaultReserves = totalVaultReserves - amount
            RETURN TRUE
        ELSE
            RETURN FALSE
        END IF
    END METHOD

    STATIC METHOD GetVaultSummary()
        DISPLAY "Total Branches: ", totalBranchesActive
        DISPLAY "Total Reserves: ", totalVaultReserves
    END METHOD

    STATIC METHOD GetGlobalLiquidityRatio(targetReserve)
        RETURN totalVaultReserves / targetReserve
    END METHOD

END CLASS


MAIN
```

```
    CREATE Branch1 = BranchOffice("B01", "Chennai", 500000)
    CREATE Branch2 = BranchOffice("B02", "Bangalore", 300000)

    CALL Branch1.Deposit(150000)
    CALL Branch2.Withdraw(50000)

    CALL BranchOffice.GetVaultSummary()

    ratio = BranchOffice.GetGlobalLiquidityRatio(1000000)

    DISPLAY "Liquidity Ratio: ", ratio

END MAIN
```

2. Dry-Run Trace Table

Initial reserves:

\[500000 + 300000 = 800000 \]

Operation	Branch 1	Branch 2	Total Reserves
Initial State	500000	300000	800000
Branch 1 deposits 150000	650000	300000	950000
Branch 2 withdraws 50000	650000	250000	900000

Final Output:

Total Branches: 2
Total Reserves: 900000
Liquidity Ratio: 0.9

Question 2: Matrix Mathematical Engine with Arrays in Class & Friend Dot-Product

1. Structured Pseudocode

CLASS Matrix2D

```
PRIVATE:
   grid : ARRAY[3][3] OF REAL

PUBLIC:

   DECLARE METHOD Initialize()
   DECLARE METHOD SetElement(row, col, value)
   DECLARE METHOD GetElement(row, col) RETURNS REAL
   DECLARE METHOD Display()

   FRIEND FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B)
      RETURNS REAL

END CLASS


METHOD Matrix2D::Initialize()

   FOR i = 0 TO 2
      FOR j = 0 TO 2
         grid[i][j] = 0
      END FOR
   END FOR

END METHOD


METHOD Matrix2D::SetElement(row, col, value)

   IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
      grid[row][col] = value
   ELSE
      DISPLAY "Invalid Index"
   END IF

END METHOD


METHOD Matrix2D::GetElement(row, col) RETURNS REAL

   IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
      RETURN grid[row][col]
   ELSE
```

```
        DISPLAY "Invalid Index"
        RETURN 0
    END IF

END METHOD


METHOD Matrix2D::Display()

    FOR i = 0 TO 2
        FOR j = 0 TO 2
            DISPLAY grid[i][j]
        END FOR
        DISPLAY NEWLINE
    END FOR

END METHOD


FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B)
    RETURNS REAL

    sum = 0

    FOR i = 0 TO 2
        sum = sum + (A.grid[i][i] * B.grid[i][i])
    END FOR

    RETURN sum

END FUNCTION


MAIN

    CREATE A, B

    CALL A.Initialize()
    CALL B.Initialize()

    SET A diagonal = [2, 4, 6]
    SET B diagonal = [1, 3, 5]
```

```
    result = ComputeCrossTrace(A, B)

    DISPLAY "Cross Trace: ", result

END MAIN
```

2. Dry-Run Trace Table

Given:

- Matrix A diagonal = [2, 4, 6]
- Matrix B diagonal = [1, 3, 5]

Calculation:

$$ (2 \times 1) + (4 \times 3) + (6 \times 5) $$
$$ = 2 + 12 + 30 = 44 $$

Iteration	A[i][i]	B[i][i]	Product	Sum
0	2	1	2	2
1	4	3	12	14
2	6	5	30	44

Output:

Cross Trace: 44

Question 3: Student Academic Roster via Array of Objects

1. Structured Pseudocode

```
CLASS StudentRecord

PRIVATE:
    rollNo : INTEGER
    studentName : STRING
    marks : ARRAY[3] OF REAL
    percentage : REAL
    grade : CHARACTER
```

```
    METHOD CalculateResult()

        percentage = (marks[0] + marks[1] + marks[2]) / 3

        IF percentage >= 85 THEN
            grade = 'A'
        ELSE IF percentage >= 70 THEN
            grade = 'B'
        ELSE IF percentage >= 50 THEN
            grade = 'C'
        ELSE
            grade = 'F'
        END IF

    END METHOD

PUBLIC:

    CONSTRUCTOR()
        rollNo = 0
        studentName = ""
        marks = [0, 0, 0]
        percentage = 0
        grade = 'F'
    END CONSTRUCTOR

    METHOD ReadData(r, name, m1, m2, m3)

        rollNo = r
        studentName = name
        marks = [m1, m2, m3]

        CALL CalculateResult()

    END METHOD

    METHOD GetPercentage() RETURNS REAL
        RETURN percentage
    END METHOD

    METHOD Display()

        DISPLAY rollNo, studentName
```

```
        DISPLAY marks
        DISPLAY percentage
        DISPLAY grade

    END METHOD

END CLASS


MAIN

    DECLARE StudentRecord students[3]
    DECLARE temp AS StudentRecord

    CALL students[0].ReadData(101, "Arun", 90, 85, 95)
    CALL students[1].ReadData(102, "Bala", 70, 75, 80)
    CALL students[2].ReadData(103, "Charan", 60, 65, 55)

    FOR i = 0 TO 1

        FOR j = 0 TO 1 - i

            IF students[j].GetPercentage() <
               students[j+1].GetPercentage() THEN

                temp = students[j]
                students[j] = students[j+1]
                students[j+1] = temp

            END IF

        END FOR

    END FOR

    DISPLAY "Student Rank List"

    FOR i = 0 TO 2
        DISPLAY "Rank: ", i + 1
        CALL students[i].Display()
    END FOR

END MAIN
```

2. Dry-Run Trace Table

Initial percentages:

- Arun = 90%
- Bala = 75%
- Charan = 60%

Bubble Sort – Descending Order

Pass	Comparison	Action
1	Arun (90) and Bala (75)	No swap
1	Bala (75) and Charan (60)	No swap
2	Arun (90) and Bala (75)	No swap

Final sorted order:

Rank	Student	Percentage	Grade
1	Arun	90%	A
2	Bala	75%	B
3	Charan	60%	C

Output:

Student Rank List

Rank: 1
Roll No: 101
Name: Arun
Percentage: 90
Grade: A

Rank: 2

Roll No: 102
Name: Bala
Percentage: 75
Grade: B

Rank: 3
Roll No: 103
Name: Charan
Percentage: 60
Grade: C

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

1. Structured Pseudocode

Important: The original signatures have an ambiguity because both the one-argument overload and the overload with default arguments accept LogEvent(message). To make the pseudocode work correctly, the severity argument is made compulsory in the second overload.

```
INLINE FUNCTION FormatPrefix(prefixTag AS STRING)
   RETURNS STRING

   RETURN "[" + prefixTag + "] "

END FUNCTION


FUNCTION LogEvent(message AS STRING)

   prefix = FormatPrefix("INFO")

   DISPLAY prefix + message

END FUNCTION


FUNCTION LogEvent(message AS STRING,
                  severityLevel AS INTEGER,
                  timestamp AS STRING = "00:00:00")

   IF severityLevel = 1 THEN
```

```
        severity = "LOW"
    ELSE IF severityLevel = 2 THEN
        severity = "MEDIUM"
    ELSE IF severityLevel = 3 THEN
        severity = "CRITICAL"
    END IF

    DISPLAY "[" + severity + "] @ " +
            timestamp + " : " + message

END FUNCTION


FUNCTION LogEvent(errorCode AS INTEGER,
                  systemModule AS STRING)

    DISPLAY "[HARDWARE_FAULT] Module: " +
            systemModule + " -> Error Code: " +
            HEX(errorCode)

END FUNCTION


MAIN

    CALL LogEvent("System Started")

    CALL LogEvent("Low Battery", 1)

    CALL LogEvent("System Failure", 3, "10:30:00")

    CALL LogEvent(255, "CPU")

END MAIN
```

2. Function Calls and Dry Run

Function Call	Arguments	Default Argument Used
LogEvent("System Started")	1	INFO prefix

LogEvent("Low Battery", 1)	2	Timestamp = 00:00:00
LogEvent("System Failure", 3, "10:30:00")	3	No default needed
LogEvent(255, "CPU")	2	Hardware fault overload

Output:

[INFO] System Started

[LOW] @ 00:00:00 : Low Battery

[CRITICAL] @ 10:30:00 : System Failure

[HARDWARE_FAULT] Module: CPU -> Error Code: 0xFF

Question 5: Hardware Register Bank with Return-by-Reference

1. Structured Pseudocode

```
CLASS RegisterBank

PRIVATE:
   registers : ARRAY[8] OF INTEGER
   errorSink : INTEGER

PUBLIC:

   CONSTRUCTOR()

      FOR i = 0 TO 7
         registers[i] = 0
      END FOR

      errorSink = -1

   END CONSTRUCTOR


   METHOD GetRegister(index AS INTEGER)
```

```
        RETURNS REF INTEGER

        IF index >= 0 AND index < 8 THEN

            RETURN REF registers[index]

        ELSE

            DISPLAY "Error: Invalid Register Index"

            RETURN REF errorSink

        END IF

    END METHOD


    METHOD DisplayState()

        DISPLAY "Register Bank State"

        FOR i = 0 TO 7
            DISPLAY HEX(registers[i])
        END FOR

    END METHOD

END CLASS


MAIN

    CREATE bank AS RegisterBank

    bank.GetRegister(0) <- 170
    bank.GetRegister(1) <- 255

    readVal <- bank.GetRegister(0)

    DISPLAY "Register 0 Value: ", HEX(readVal)

    bank.GetRegister(10) <- 999
```

```
    CALL bank.DisplayState()

END MAIN
```

2. Explain how return-by-reference enables a function call to appear on the left side of an assignment.

- Return-by-reference returns the actual memory location of a variable instead of returning a copy.
- The function returns an alias to the original variable.
- Therefore, the function call behaves like an lvalue.
- It allows direct modification of the original variable using assignment.

Example:

```
bank.GetRegister(0) <- 170
```

Here, GetRegister(0) returns a reference to the first register. Therefore, the value 170 is directly stored in that register.

For an invalid index, the function returns a reference to errorSink instead of accessing an invalid memory location.

Dry-Run Trace Table

Operation	Register 0	Register 1	Error Sink
Initial state	0	0	-1
Assign 170	170	0	-1
Assign 255	170	255	-1
Read Register 0	170	255	-1
Invalid index 10 assigned 999	170	255	999

Output:

```
Register 0 Value: 0xAA
```

Error: Invalid Register Index

Register Bank State
0xAA
0xFF
0x0
0x0
0x0
0x0
0x0
0x0